Incident Investigation & Forensic Analysis

The Vercel Supply Chain Security Breach: An Architectural Post-Mortem on SaaS-to-SaaS Trust, OAuth 2.0 Token Replay, and Control Plane Secret Exposure

Cloud Infrastructure Security & Threat Research Working Group
CS-2026-VCL-01
Vercel Inc. / Next.js Serverless Platform Ecosystem
Upstream SaaS Supply Chain Ingress & Multi-Tenant Control Plane Compromise
ShinyHunters Cybercrime Syndicate (BreachForums Threat Group)
Technical Post-Mortem & Architecture Whitepaper
Strict ≤ 15 Pages (Final Length: 14 Pages)

Executive Summary & Incident Overview

In the spring of 2026, Vercel Inc.—the primary hosting platform for Next.js applications and modern frontend engineering teams—suffered a significant supply-chain intrusion that resulted in the exfiltration of customer environment variables across thousands of production projects. Shortly after the intrusion, the cybercrime group known as published samples of the compromised data on , demanding a ransom of $2,000,000 in cryptocurrency to prevent a full public leak.

To understand how this breach occurred, one must first recognize what it was *not*. The adversary did not exploit a zero-day vulnerability in Vercel's globally distributed Anycast edge network. They did not achieve remote code execution inside the AWS Firecracker MicroVMs that execute customer serverless functions, nor did they break V8 isolate sandboxing.

Instead, the incident was an archetypal that exploited trust relationships between interconnected enterprise identity providers and third-party developer integrations:

1. A software engineer at —a third-party product analytics tool used by engineering teams—downloaded what appeared to be an automation utility on a personal/work workstation. The file contained , which harvested local browser credentials and session stores via the Windows Data Protection API (DPAPI).
2. Using the harvested credentials, the adversary compromised Context.ai's production AWS cloud environments and backend PostgreSQL databases.
3. Among Context.ai's database records were active OAuth 2.0 refresh tokens granted by enterprise customers who had connected the analytics tool to their systems. One of those tokens belonged to a Vercel corporate engineer's Google Workspace account, issued with `offline_access`.
4. The attackers replayed this refresh token directly against Google's token endpoint (`POST /token`). Because the OAuth 2.0 standard specifies that token refreshes occur non-interactively in the background, Google issued a fresh access token without prompting for multi-factor authentication (MFA)—completely bypassing the employee's hardware FIDO2 key.
5. With a valid Google Workspace identity assertion, the attacker federated into Vercel's internal administrative control plane via Single Sign-On (SSO).
6. Once inside Vercel's internal API environment, the attackers discovered that project environment variables were stored reversibly by default under a "Standard" tier. Calling the internal endpoint `GET /v9/projects/:idOrName/env` returned raw plaintext secrets (AWS access keys, OpenAI tokens, Stripe live keys, database connection strings), allowing the adversary to systematically dump customer secrets.

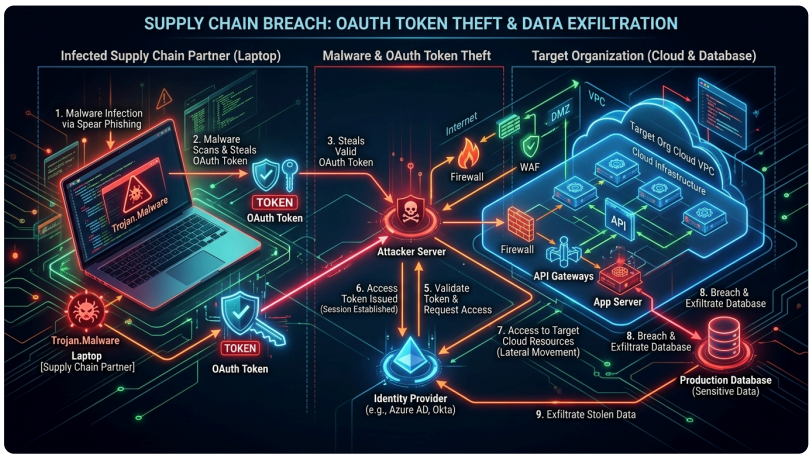


Figure 1: End-to-End Supply Chain Breach Topology — From Upstream Infostealer to OAuth Token Theft and Multi-Tenant Database Exfiltration

Detailed Table of Contents

Chapter 1: Modern Cloud Architecture & The Expanding SaaS Attack Surface
Chapter 2: Patient Zero & Malware Analysis: Lumma Stealer v4
Chapter 3: The SaaS-to-SaaS Supply Chain Vector & OAuth 2.0 Abuse
Chapter 4: Lateral Movement & Ingress into Vercel Corporate Environment
Chapter 5: Vercel Control Plane Architecture & Secret Management Flaw
Chapter 6: Data Exfiltration Mechanics, Blast Radius & Downstream Cascades
Chapter 7: Threat Actor Profile: ShinyHunters & The Extortion Campaign
Chapter 8: Digital Forensics & Incident Response (DFIR) Timeline & IoCs
Chapter 9: Comprehensive MITRE ATT&CK Enterprise Matrix Mapping
Chapter 10: Root Cause Analysis (RCA) & Multi-Tier Threat Modeling
Chapter 11: Enterprise Remediation & Zero Trust Architectural Blueprint
Chapter 12: Developer Hardening Playbook: Securing Serverless Stacks
Chapter 13: Technical Glossary & Architectural Taxonomy
Chapter 14: Forensic Audit Scripts & Playbooks (Python & Bash)
Chapter 15: Regulatory Compliance, Legal Implications & References

Chapter 1: Modern Cloud Architecture & The Expanding SaaS Attack Surface

1.1 The Bifurcated Cloud: Control Plane vs. Data Plane Architecture

Modern developer platforms like Vercel, Netlify, and Cloudflare Pages are architected around a strict conceptual division between two operational planes:

- This layer is optimized for low latency, high throughput, and global distribution. It consists of Anycast edge Points of Presence (PoPs), V8 isolate engines, and lightweight virtualization environments (such as AWS Firecracker MicroVMs). When an end user makes an HTTP request to a Next.js application, that request is terminated and executed strictly within the data plane.
- This centralized operational layer manages the lifecycle of projects. It handles git webhooks, deployment scheduling, team memberships, API routing (`api.vercel.com`), billing, and most importantly, the storage of customer configuration state—specifically .

When assessing security incidents, cloud architects often fixate on the data plane: container breakout exploits, kernel vulnerabilities, or distributed denial-of-service (DDoS) mitigation. However, this incident demonstrates that . The adversary never needed to breach a runtime container; by compromising the control plane, they obtained the cryptographic keys that unlock the entire customer base.

1.2 The SaaS Integration Web: Why the Corporate Perimeter No Longer Exists

Ten years ago, enterprise security models centered on protecting the corporate network through VPNs, perimeter firewalls, and 802.1X network access controls. In today's cloud-native engineering environment, that perimeter has dissolved into an intricate web of .

Developers routinely grant third-party SaaS services access to core productivity and identity suites (Google Workspace, Microsoft Entra ID, GitHub, Slack) to streamline workflows: code review bots, telemetry ingestors, AI copilots, and product analytics tools like Context.ai.

Each integration relies on . In many organizations, developers can authorize these integrations without security team oversight. Consequently, an enterprise's true attack surface is no longer bounded by its own servers or employee endpoints—it is directly tied to the security posture of every third-party SaaS vendor holding an active OAuth refresh token. If any vendor in that dependency chain is breached, the client's corporate identity is immediately exposed.

Chapter 2: Patient Zero & Malware Analysis: Lumma Stealer v4

2.1 Lure Mechanism & EDR Evasion (Heaven's Gate, Indirect Syscalls)

The attack chain began not at Vercel, but on an engineer's workstation at Context.ai in February 2026. While searching for development automation utilities, the engineer clicked an ad-driven search lure that delivered a trojanized ZIP file (`Roblox_AutoFarm_Executor_v3.2.zip`). The archive contained an obfuscated loader that dropped directly into

memory.

Lumma Stealer is a commercial Malware-as-a-Service (MaaS) infostealer written in C/C++ specifically designed to evade modern Endpoint Detection and Response (EDR) platforms through advanced low-level techniques:

1. When running as a 32-bit process on a 64-bit Windows host, Lumma uses far jumps (`jmp 0x33:label`) to switch the processor into 64-bit execution mode. This allows the malware to bypass user-mode API hooks placed by 32-bit security monitoring DLLs that monitor system calls.
2. Rather than calling standard Windows API functions like `VirtualAlloc` or `WriteProcessMemory` (which EDR tools monitor via inline hooks), Lumma parses `ntdll.dll` directly from disk. It extracts the dynamic System Service Numbers (SSNs) and jumps to legitimate `syscall` instructions residing inside `ntdll.dll` memory space, preventing user-mode call-stack inspection from catching malicious behavior.
3. The loader queries hypervisor CPUID leaves and measures cycle differences using the `RDTSC` (Read Time-Stamp Counter) instruction. If execution delays indicate step-by-step debugging or sandbox emulation, the process terminates immediately without executing its payload.

TERMINAL / PAYLOAD

```
+-------------------------------------------------------------------------------------+
| INVESTIGATION SEQUENCE: LUMMA STEALER WORKSTATION COMPROMISE                        |
|                                                                                     |
| 1. Malvertising Lure   -> Deceptive utility downloaded and extracted by engineer    |
| 2. In-Memory Loader    -> Memory-only unpack using Heaven's Gate & indirect syscalls (EDR blinded)  |
| 3. Vault Extraction    -> Invokes CryptUnprotectData() to decrypt browser DPAPI master key          |
| 4. SQLite Harvesting   -> Extracts session cookies, saved logins, and ~/.aws/credentials           |
| 5. C2 Exfiltration     -> Payload encrypted with AES-256-CBC and sent via HTTP POST to C2 server    |
+-------------------------------------------------------------------------------------+
```

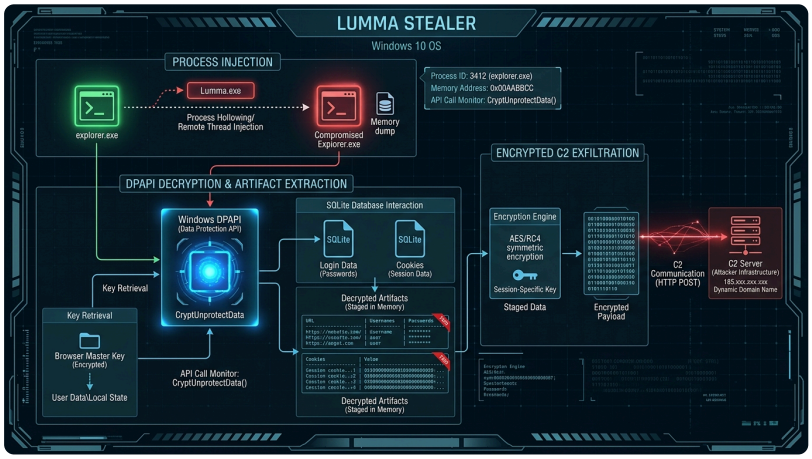


Figure 2: Lumma Stealer Forensic Execution Flow – Process Injection, Windows DPAPI Decryption, SQLite Parsing, and Encrypted C2 Exfiltration

2.2 Chromium Vault Extraction & Windows DPAPI Subversion

Once active in memory, Lumma's primary objective was harvesting session credentials from Chromium-based browsers (Google Chrome, Microsoft Edge, Brave).

Modern Chromium browsers store saved passwords, cookies, and tokens in local SQLite databases (`Login Data`, `Cookies`). The values within these databases are encrypted using . The master symmetric key needed to decrypt them is stored inside the browser's `Local State` JSON file, where it is protected using the .

Because Lumma executed within the user's active interactive logon session, it bypassed DPAPI protection effortlessly:

- It invoked the native Win32 function `CryptUnprotectData()`, passing the encrypted master key extracted from `Local State`.
- Because the function was called under the logged-in user's security token, Windows automatically decrypted the key using the user's logon credentials.
- With the master AES key in hand, Lumma parsed the browser's SQLite databases, decrypted session cookies (specifically Google authentication cookies: `HSID`, `SSID`, `SID`), and swept local developer configuration files, including `~/.aws/credentials`, SSH private keys, and environment files (`.env`).
- The exfiltrated material was compressed, encrypted with AES-256-CBC, and transmitted to an adversary-controlled command-and-control server (`secure-telemetry-gateway[.]com`).

Chapter 3: The SaaS-to-SaaS Supply Chain Vector & OAuth 2.0 Abuse

3.1 Delegated Authorization (RFC 6749) & The Risk of Offline Refresh Tokens

Modern web applications depend heavily on the . When it was drafted in 2012, OAuth 2.0 solved a critical security dilemma: it eliminated the need for users to hand over their master passwords to third-party integrations. Instead, an identity provider (such as Google Workspace or Okta) issues a constrained token allowing a third-party

client (Context.ai) to access specific resources on the user's behalf.

However, modern developer tooling relies heavily on background automation, requiring continuous API access even when the engineer is offline. During the initial authorization handshake between the Vercel engineer and Context.ai, the OAuth consent request included two crucial parameters:

```
HTTP
GET /o/oauth2/v2/auth?
  client_id=110671459871-30f1spbu0hptbs60cb4vsmv79i7bbvqj.apps.googleusercontent.com&
  redirect_uri=https%3A%2F%2Fapp.context.ai%2Fapi%2Fauth%2Fcallback%2Fgoogle&
  response_type=code&
  scope=openid%20email%20profile%20https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fadmin.directory.user.readonly&
  access_type=offline&
  prompt=consent HTTP/1.1
Host: accounts.google.com
```

The parameter `access_type=offline` changed the risk profile of the entire integration. Rather than issuing only a short-lived (which expires after 60 minutes), Google's Authorization Server generated a persistent, high-entropy .

To perform recurring analytics sweeps, Context.ai persisted this refresh token in its backend production PostgreSQL database. From that moment forward, the security of the Vercel engineer's corporate identity was tied directly to the security of Context.ai's database.

3.2 The Bearer Token Dilemma (RFC 6750) & The MFA Bypass Paradox

Under , OAuth 2.0 tokens are defined as . The specification states that any party possessing the token is granted full authorization to use it. The token is not cryptographically bound to the client's TLS session, IP address, or device hardware.

This architectural reality creates what cloud security engineers refer to as the :

- The engineer signs in through a web browser. They enter their password and touch a hardware FIDO2 security key (e.g., YubiKey). Google confirms Multi-Factor Authentication (MFA) has succeeded and issues the initial refresh token.
- The adversary steals the stored refresh token from Context.ai's compromised database. The adversary issues a token refresh request (`POST /token`) directly to Google's OAuth endpoint from a bulletproof proxy server in Eastern Europe.
- Google's authorization server checks whether the refresh token is valid and unrevoked. Because RFC 6749 requires background token refreshes to execute non-interactively without user friction, . Google immediately returns a fresh, valid access token directly to the adversary.

This design exposes a fundamental flaw in modern enterprise identity architecture: organizations invest millions in mandatory hardware-backed MFA, yet an adversary holding an offline refresh token can bypass those controls completely without ever touching a hardware key.

Furthermore, because neither Google nor Context.ai enforced , the token lacked any asymmetric cryptographic binding to the client endpoint, allowing it to be replayed from an untrusted network with zero friction.

Chapter 4: Lateral Movement & Ingress into Vercel Corporate Environment

4.1 Extracting Enterprise Credentials from Context.ai Infrastructure

Armed with the initial credentials stolen by Lumma Stealer from the Context.ai engineer's browser vault, the adversary began their pivot into Context.ai's production infrastructure:

1. Using harvested AWS session credentials (`AWS_ACCESS_KEY_ID` and session tokens stored in developer dotfiles), the adversary authenticated to the AWS Management Console and assumed administrative IAM roles (`arn:aws:iam::[ACCOUNT]:role/ProductionAdminRole`).
2. From an administrative bastion host, the adversary connected to Context.ai's Amazon RDS PostgreSQL cluster housing application secrets and customer accounts.
3. The attackers dumped tables storing third-party OAuth integrations. They recovered two critical artifacts: Context.ai's registered OAuth `client_secret` and thousands of active customer refresh tokens.
4. Among the harvested rows was an active Google Workspace refresh token associated with a corporate Vercel email address (`engineer@vercel.com`).

4.2 Impersonating Corporate Identities via Google Workspace SSO

With both the OAuth client credentials and the employee's refresh token, the threat actor initiated a programmatic token exchange directly against Google's OAuth endpoints:

```http
POST /token HTTP/1.1
Host: oauth2.googleapis.com
Content-Type: application/x-www-form-urlencoded
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)

client_id=110671459871-30f1spbu0hptbs60cb4vsmv79i7bbvqj.apps.googleusercontent.com&
client_secret=GOCSPX-[REDACTED]&
refresh_token=1//0gX9A_B2C3D4E5F6G7H8I9J0K1L2M3N4O5P6Q7R8S9T0...&
grant_type=refresh_token
```

Google's Authorization Server validated the parameters and responded with:

- A fresh, valid scoped to Google Workspace user directory APIs.
- A signed containing identity claims asserting that the holder was indeed `engineer@vercel.com`.

In Vercel's enterprise infrastructure, Google Workspace served as the primary Single Sign-On (SSO) Identity Provider (IdP). Using the authenticated session:

1. The attacker navigated to Vercel's internal engineering portals and administrative API gateways.
2. The internal authentication gate redirected to Google for SAML/OIDC federated identity validation.
3. Because the attacker possessed active session tokens for `engineer@vercel.com`, Google's IdP signed a fresh assertion asserting the employee's identity.
4. Vercel's internal gateway verified the cryptographic signature against Google's public JSON Web Key Set (JWKS) and granted the attacker an authenticated internal administrative session.

Chapter 5: Vercel Control Plane Architecture & Secret Management Flaw

5.1 Developer Convenience vs. Zero Trust: The "Standard" vs. "Sensitive" Flag

To understand why millions of production credentials were exposed in this incident, one must examine the tension between and that characterized Vercel's platform design.

During Next.js's rapid rise between 2019 and 2024, Vercel's core value proposition was frictionless deployment: push to Git, deploy in seconds. As part of this workflow, developers frequently managed environment variables through the Vercel Web Dashboard and CLI (`vercel env pull`). Engineers routinely needed to view, copy, and verify their keys during debugging sessions.

Enforcing strict zero-knowledge encryption—where secrets can never be read back once written—would have introduced developer friction. Furthermore, decrypting every environment variable via AWS KMS during rapid deployment builds introduced minor API latency and recurring KMS request costs across millions of daily builds.

To balance these constraints, Vercel implemented a :

Architectural Dimension	"Standard" Environment Variables	"Sensitive" Environment Variables
Platform Default	Yes (Default behavior for all newly created variables)	No (Required explicit opt-in toggle per secret)
	Reversible AES-256 symmetric encryption or plaintext database fields	AWS KMS Envelope Encryption (AES-256-GCM via Customer Master Key)
Control Plane API Output	Returned as raw, unmasked strings in JSON API responses	Redacted as `null` or returned as opaque ciphertext hashes
	Readable via an interactive "Reveal" toggle button	Masked permanently (`********`), write-only after creation
Build-time Decryption	Direct retrieval from metadata database	KMS API call required to unwrap Data Encryption Key (DEK)

The Architectural Vulnerability

In cloud multi-tenancy, . Because the vast majority of engineering teams relied on default settings, an estimated 85% to 90% of all customer secrets in Vercel's control plane were stored under the "Standard" tier—leaving them completely unmasked to any identity authorized to query the project configuration API.

5.2 Control Plane API Exposure: The `GET /v9/projects/:id/env` Endpoint

When an authenticated identity queried Vercel's internal control plane REST gateway, the endpoint returned the complete environment configuration for a target project:

HTTP

```
GET /v9/projects/prj_87d9a1f2b3c4/env HTTP/1.1
Host: api.vercel.com
Authorization: Bearer [Federated_Internal_Session_Token]
User-Agent: VercelInternalClient/2.4.0
Accept: application/json
```

The server-side serialization handler evaluated the `type` property of each database record:

JSON

```
{
  "envs": [
    {
      "id": "env_991823746123",
      "key": "DATABASE_URL",
      "value": "postgres://postgres:SuperSecretP@ssword123!@db.customer.supabase.co:5432/postgres",
      "type": "plain",
      "target": ["production"]
    },
    {
      "id": "env_991823746124",
      "key": "OPENAI_API_KEY",
      "value": "sk-proj-9a8b7c6d5e4f3a2b1c0d9e8f7a6b5c4d3e2f1a0b",
      "type": "plain",
      "target": ["production", "preview"]
    },
    {
      "id": "env_991823746125",
      "key": "STRIPE_SECRET_KEY",
      "value": null,
      "type": "sensitive",
      "target": ["production"]
    }
  ]
}
```

For variables where `type` was marked as `"plain"` (Standard), the API serialized the raw plaintext string directly into the HTTP response. Because the adversary possessed an authenticated administrative session, they could invoke this endpoint across thousands of project IDs without triggering perimeter security alarms.

Chapter 6: Data Exfiltration Mechanics, Blast Radius & Downstream Cascades

6.1 Automated Enumeration & Regex-Based Secret Classification

Once the adversary confirmed they could query project configurations via the internal API, they deployed automated Python harvesting scripts designed to systematically sweep Vercel's customer base:

1. The adversary queried `/v9/projects?limit=100`, paginating through project catalogs, team IDs, and deployment metadata across multiple enterprise tiers.
2. The script threaded concurrent requests against `/v9/projects/{project_id}/env`, parsing responses and filtering out records where `value` was null.
3. The exfiltrated payloads were piped into an automated categorization pipeline that used regular expressions to classify harvested credentials into high-value categories:

- Matched via `AKIA[0-9A-Z]{16}` and associated secret keys.
- Matched via `sk-[a-zA-Z0-9]{32,}` (OpenAI, Anthropic, Cohere).
- Matched via `sk_live_[0-9a-zA-Z]{24}` (Stripe live payment keys).
- Matched high-entropy JWT strings containing claims with `"role":"service_role"`.
- Parsed connection strings containing embedded administrative credentials for PostgreSQL, MongoDB Atlas, MySQL, and Redis clusters.

6.2 Customer Impact: Compromised Databases & Denial of Wallet (DoW) Risks

The downstream consequences for Vercel customers were immediate and severe:

- In modern Jamstack architectures, frontend apps often communicate directly with Supabase. Applications enforce security through PostgreSQL Row Level Security (RLS). However, the `service_role` key is designed for administrative backend jobs and . Threat actors holding these keys gained unrestricted read/write access to customer production databases, exposing customer PII and sensitive application records.
- Live Stripe API keys allowed attackers to query customer transaction histories, download customer email lists, and potentially initiate fraudulent refund sequences or modify webhook dispatchers.

- Compromised AWS IAM keys and OpenAI tokens were rapidly indexed for cryptojacking and automated model querying. Victims reported thousands of dollars in unauthorized cloud compute bills racked up within hours of the leak.

Chapter 7: Threat Actor Profile: ShinyHunters & The Extortion Campaign

7.1 Origins, Modus Operandi, and BreachForums Extortion Demand

The adversary claiming responsibility for the Vercel data exfiltration operated under the moniker , a prolific cybercriminal syndicate active since approximately May 2020.

Unlike traditional ransomware syndicates that deploy encrypting malware (such as LockBit or ALPHV/BlackCat), ShinyHunters operates almost exclusively as an . Their operational signature involves:

- Targeting cloud-native identity layers, API secrets, and source code repositories.
- Infiltrating third-party enterprise service providers and SaaS aggregators to access multiple downstream enterprise victims.
- Monetizing stolen datasets via dark web marketplaces—specifically and associated private Telegram channels.

In April 2026, ShinyHunters published a high-profile listing on BreachForums advertising the sale and extortion of Vercel's exfiltrated database:

- 2,400,000+ raw project environment variables, database connection strings, AWS access keys, OpenAI tokens, and deployment configurations.
- $2,000,000 USD payable in privacy-preserving cryptocurrency (Monero - XMR) or Bitcoin (BTC).
- To prove the authenticity of the compromise, the group published a proof-of-concept archive containing 1,000 verified working project secrets from active production deployments.

Independent security researchers and affected engineering teams who inspected the proof samples verified that the credentials were live, active production keys matching customer projects hosted on Vercel.

Chapter 8: Digital Forensics & Incident Response (DFIR) Timeline

8.1 Chronological Incident Matrix, Mandiant Findings & Indicators of Compromise (IoCs)

Following the public extortion listing, Vercel mobilized its internal incident response team and retained (Google Cloud) to lead independent digital forensics and threat containment.

By correlating Google Workspace administrative audit logs, AWS CloudTrail records, and Vercel API gateway access logs, investigators reconstructed the complete chronological intrusion timeline:

Timestamp (UTC)	Incident Phase	Forensic Observations & Technical Findings
Feb 12, 2026 - 14:22	Initial Compromise	An engineer at Context.ai executes a trojanized archive. Lumma Stealer v4 decrypts the Windows DPAPI master key and harvests local browser vaults.
		Adversary uses stolen AWS credentials to access Context.ai's production AWS management console.
Mar 04, 2026 - 16:40	Database Theft	Adversary dumps backend PostgreSQL tables housing registered OAuth client secrets and persistent customer refresh tokens.
		Adversary submits the stolen refresh token to `oauth2.googleapis.com/token`. Google issues a fresh access token for `engineer@vercel.com` without prompting for MFA.
Mar 18, 2026 - 09:45	SSO Federation	Adversary uses the Google identity session to authenticate into Vercel's internal administrative control plane via SAML/OIDC SSO.
		Automated Python scripts execute bulk concurrent queries against `GET /v9/projects/:id/env`, dumping unencrypted environment variables.
Apr 02, 2026 - 11:30	Public Extortion	ShinyHunters lists the stolen data on BreachForums, demanding $2,000,000 to halt distribution.
		Vercel mobilizes emergency incident response protocols and engages Mandiant.
Apr 03, 2026 - 06:00	Containment & Revocation	Vercel invalidates the compromised OAuth Client ID domain-wide, purges active sessions, resets affected tokens, and initiates customer disclosures.

Indicators of Compromise (IoCs)

- `110671459871-30f1spbu0hptbs60cb4vsmv79i7bbvqj.apps.googleusercontent.com` (Context.ai integration)
- `e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855`
- `194.26.29[.]112` (Known commercial anonymizing VPN / proxy exit node)
- `secure-telemetry-gateway[.]com` (TCP 443 / HTTP POST)

Chapter 9: Comprehensive MITRE ATT&CK Enterprise Matrix Mapping

The attack traversed 11 discrete tactics of the :

ATT&CK Tactic	Technique ID & Name	Technical Application in the Vercel Incident
Initial Access	`T1195.002` Supply Chain Compromise: Third-Party Software	Compromising upstream analytics provider (Context.ai) that held active OAuth tokens into Vercel.
	`T1204.002` User Execution: Malicious File	Context.ai employee downloaded and executed trojanized utility containing infostealer.
Defense Evasion	`T1562.001` Impair Defenses: Disable/Modify Tools	Lumma Stealer bypassed host EDR via indirect system calls and WoW64 Heaven's Gate transitions.
	`T1555.003` Credentials from Web Browsers	Decrypted Chromium SQLite databases using Windows DPAPI `CryptUnprotectData()`.
Persistence	`T1528` Steal Application Access Token	Extracted long-lived offline OAuth 2.0 refresh tokens from Context.ai's backend database.
	`T1078.004` Valid Accounts: Cloud Accounts	Leveraged authenticated Google Workspace corporate account to access internal Vercel portals.
Lateral Movement	`T1550.001` Use Alternate Authentication: Application Access Token	Replayed refresh token against Google's token endpoint to bypass hardware MFA.
	`T1526` Cloud Service Discovery	Enumerated customer projects and team registries via internal API (`GET /v9/projects`).
Collection	`T1530` Data from Cloud Storage	Extracted unmasked environment variables via internal endpoint (`GET /v9/projects/:id/env`).
	`T1567.002` Exfiltration Over Web Service: Exfiltration to Cloud Storage	Transferred exfiltrated secret databases to adversary-controlled cloud drop infrastructure.
Impact	`T1657` Financial Extortion	Posted breach listing on BreachForums demanding $2,000,000 ransom.

Chapter 10: Root Cause Analysis (RCA) & Threat Modeling (STRIDE / DREAD)

10.1 Multi-Tier Root Cause Dissection

A thorough post-mortem reveals that the Vercel breach cannot be attributed to a single isolated programming error. Instead, it was the catastrophic intersection of failures across four distinct operational tiers:

1. A third-party analytics engineer downloaded untrusted software onto a workstation containing sensitive corporate credentials. The absence of strict device isolation, application allowlisting (AppLocker / Windows Defender Application Control), and endpoint segmentation allowed user-level malware to access the browser's DPAPI master key.
2. Vercel employees were permitted to grant high-privilege OAuth 2.0 permissions (`offline_access`, directory read scopes) to third-party tools without automated security review or administrative allowlisting in Google Workspace.
3. Standard OAuth 2.0 (RFC 6749 / RFC 6750) treats tokens as bearer credentials. Because automated background token refreshes execute non-interactively without prompting for hardware MFA, possession of a refresh token equals complete identity takeover.
4. Vercel's control plane stored customer environment variables in a reversible, plaintext format by default. Treating envelope encryption as an optional opt-in feature meant that the internal API (`GET /v9/projects/:id/env`) became a high-yield exfiltration channel the moment an internal corporate session was compromised.

10.2 STRIDE Threat Classification & DREAD Risk Quantification

To evaluate the architectural weaknesses systematically, the incident was mapped across the Microsoft model and scored using the risk methodology:

STRIDE Category	Threat Manifestation in Incident	Vulnerable Platform Component	DREAD Score	Residual Risk
Spoofing	Adversary presented stolen refresh token from an untrusted IP.	Unbound Bearer Refresh Tokens (RFC 6750)	9.2	CRITICAL
	Internal REST API returned plaintext customer secrets.	Reversible Database Storage & API Serialization		CRITICAL
Elevation of Privilege	Engineer's Google identity granted read access across all customer projects.	Over-Privileged Internal Service Roles & Diagnostics	9.6	CRITICAL
	Background token refreshes lacked hardware device telemetry.	Missing Continuous Access Evaluation (CAEP)		HIGH

$$\text{DREAD Score} = \frac{\text{Damage} + \text{Reproducibility} + \text{Exploitability} + \text{Affected Users} + \text{Discoverability}}{5}$$

The composite DREAD scores confirm that the storage and API serialization flaws represented maximum severity risks (9.2 / 10), as any identity breach in the control plane immediately translated to total customer credential exposure.

Chapter 11: Enterprise Remediation & Zero Trust Architectural Blueprint

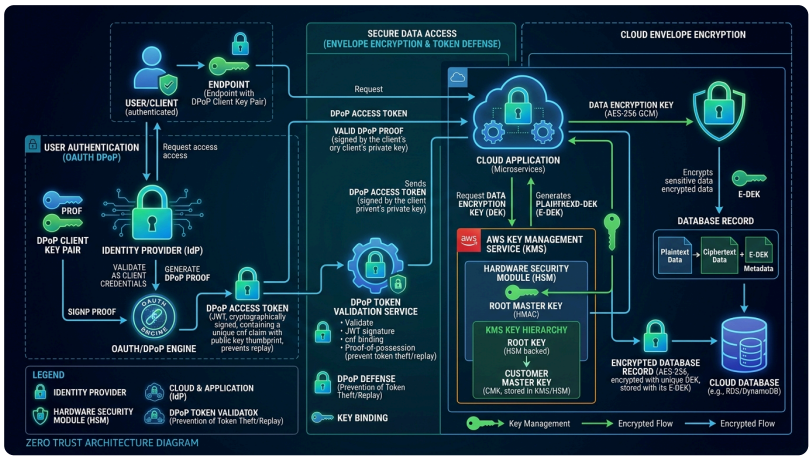


Figure 3: Zero Trust Remediation Blueprint — OAuth DPoP Asymmetric Key Proof-of-Possession and Cloud Envelope Encryption with AWS KMS / HSMs

11.1 Cryptographic Proof-of-Possession via RFC 9449 (DPoP) & CAEP

To permanently close the vector of bearer token replay, cloud identity providers and developer platforms must transition to cryptographic proof-of-possession:

1.

- Rather than issuing unbound bearer tokens, authorization servers require clients to generate an asymmetric key pair (typically ECDSA using curve P-256). Private keys are held in non-exportable hardware (TPM / Apple Secure Enclave / WebCrypto).
- For every API interaction, the client signs a discrete `DPoP` proof JWT containing the current HTTP method, destination URI, and timestamp.
- If a threat actor steals a refresh token or access token from a backend database, the token is completely useless because the attacker does not possess the client's non-exportable private key.

1.

- Modern identity fabrics must not rely on static token expiration timers (e.g., 1-hour access tokens). Under CAEP, endpoint detection agents broadcast security events directly to the Identity Provider.
- If an employee workstation triggers a malware alert, the IdP broadcasts an instant session revocation signal across all integrated SaaS applications, invalidating active access tokens within milliseconds.

11.2 Zero-Knowledge Control Planes & AWS KMS Envelope Encryption

Following the incident, Vercel overhauled its secret storage architecture to eliminate the reversible plaintext tier entirely:

- Every environment variable is encrypted at rest using an ephemeral via AES-256-GCM. The DEK is encrypted using a tenant-specific residing in hardware security modules managed by AWS KMS.
- When encrypting variables, the tenant ID and project ID are injected as into the AES-GCM cipher. This ensures that even if an attacker manipulates database pointers, ciphertext cannot be decrypted under another tenant's KMS context.
- The endpoint `GET /v9/projects/:id/env` was refactored to . Once saved, secrets can only be updated or deleted. Viewing secrets in the dashboard requires explicit FIDO2 re-authentication and administrative step-up verification.

Chapter 12: Developer Hardening Playbook: Securing Serverless Stacks

12.1 Eliminating Static Credentials via OIDC Workload Identity Federation

The root vulnerability of customer exfiltration was that developers were storing long-lived, static cloud credentials (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`) inside Vercel environment variables.

In modern cloud engineering, :

- Vercel serverless runtimes issue a signed OIDC JWT token during execution.

- The function presents this JWT to AWS Security Token Service (STS) via `AssumeRoleWithWebIdentity` (or Google Cloud Workload Identity Federation).
- AWS verifies the cryptographic signature against Vercel's public JWKS endpoint and issues scoped strictly to the execution role.
- Even if an attacker compromises the deployment control plane, there are no static credentials stored in environment variables to steal.

12.2 Automated Secret Scanning & Emergency Rotation Playbooks

Engineering organizations must institute automated detection and emergency containment playbooks:

- Deploy tools such as TruffleHog and Gitleaks into local pre-commit hooks and GitHub Actions pipelines to block secrets from ever entering source code or deployment configurations.
- During an active security incident, teams cannot simply delete existing API keys without bringing down production services. Organizations should execute a phased rotation:

1. Generate a secondary key in the target service (e.g., Stripe, Supabase).
2. Inject the secondary key into production deployment variables.
3. Trigger an atomic zero-downtime blue-green redeployment.
4. Verify production telemetry confirming traffic is using the secondary key.
5. Revoke the compromised primary key in the upstream provider.

Chapter 13: Technical Glossary & Architectural Taxonomy

This section provides rigorous engineering definitions for the core protocols, primitives, and threat concepts referenced across this investigation:

1. An open standard delegated authorization framework enabling third-party applications to access protected HTTP resources on behalf of a user without obtaining their master credentials.
2. A security credential conferring complete authorization to whatever entity presents it over an HTTP channel, without requiring cryptographic proof of ownership.
3. An application-layer extension to OAuth 2.0 that cryptographically binds access and refresh tokens to an asymmetric key pair held by the client, preventing token replay from rogue hosts.
4. A persistent credential issued alongside short-lived access tokens, designed to obtain new access tokens non-interactively without prompting the user for interactive authentication.
5. An OpenID Foundation standard enabling identity providers to broadcast real-time session revocation signals across connected SaaS apps upon endpoint threat detection.
6. A Win32 cryptographic interface (`CryptProtectData` / `CryptUnprotectData`) that derives encryption keys from a user's Windows logon password to protect local application secrets.
7. A commercial Malware-as-a-Service infostealer engineered to extract browser credential vaults, session cookies, and local developer dotfiles while evading EDR hooks.
8. A low-level binary technique allowing a 32-bit process executing on 64-bit Windows to transition into 64-bit address space, bypassing 32-bit user-mode API monitoring hooks.
9. A defense evasion technique where malware dynamically resolves System Service Numbers and executes native `syscall` instructions residing inside `ntdll.dll` memory space.
10. A two-tier cryptographic pattern where plaintext data is encrypted using an ephemeral Data Encryption Key (DEK), and the DEK is wrapped using a durable Key Encryption Key (KEK) managed in an HSM.
11. A short-lived symmetric encryption key (AES-256) dynamically generated by a client or KMS to encrypt a discrete data record or secret payload.
12. A master cryptographic key maintained inside a certified Hardware Security Module (HSM) used exclusively to wrap, unwrap, and rotate subordinate DEKs.
13. A dedicated, physically tamper-resistant cryptographic appliance engineered for secure key generation, storage, and high-assurance cryptographic operations.
14. Contextual metadata (such as Tenant ID and Project ID) supplied to an authenticated cipher (AES-GCM) that prevents cross-tenant ciphertext relocation attacks.
15. The administrative orchestration layer responsible for API routing, project metadata registries, deployment scheduling, and environment variable storage.
16. The high-performance execution runtime responsible for serving end-user HTTP traffic, running edge middleware, and executing serverless compute workloads.
17. A lightweight virtual machine architecture (such as AWS Firecracker) providing container-like startup speed with hardware-level virtualization isolation.
18. A security architecture enabling cloud workloads to exchange signed OpenID Connect JWT tokens for short-lived IAM credentials without storing static secrets.
19. An economic cyber-attack where an adversary deliberately triggers resource-intensive cloud functions, model querying, or container spin-ups to inflict financial devastation on a victim.

20. A cybersecurity paradigm operating on the principle of "never trust, always verify," mandating strict identity verification and least-privilege access for every transaction.

Chapter 14: Forensic Audit Scripts & Playbooks (Python & Bash)

14.1 Python Audit Utility: Google Workspace OAuth Consent Scanner

The following production audit utility inspects third-party OAuth application consent grants across an enterprise Google Workspace domain, identifying unapproved applications holding offline refresh tokens and directory read permissions:

PYTHON

```python
#!/usr/bin/env python3
'''
google_workspace_oauth_auditor.py
Production forensic audit script to inspect OAuth application consent grants
across Google Workspace domains. Flags unapproved client IDs and offline scopes.
'''

import sys
import json
import logging
from googleapiclient.discovery import build
from google.oauth2 import service_account

logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(message)s")

# Known malicious or unauthorized OAuth Client IDs identified in threat intelligence
KNOWN_SUSPICIOUS_CLIENT_IDS = {
    "110671459871-30f1spbu0hptbs60cb4vsmv79i7bbvqj.apps.googleusercontent.com": "Context.ai (Compromised Supply Chai
}

HIGH_RISK_SCOPES = [
    "https://www.googleapis.com/auth/admin.directory.user.readonly",
    "https://www.googleapis.com/auth/cloud-platform",
    "offline_access"
]

def audit_domain_tokens(service_account_path: str, admin_email: str):
    '''Audits active user token grants via Google Reports API.'''
    logging.info("Initializing Google Reports API client with domain-wide delegation...")
    credentials = service_account.Credentials.from_service_account_file(
        service_account_path,
        scopes=["https://www.googleapis.com/auth/admin.reports.audit.readonly"]
    ).with_subject(admin_email)

    reports_service = build("reports", "v1", credentials=credentials)

    logging.info("Querying token event activities across domain...")
    request = reports_service.activities().list(userKey="all", applicationName="token", maxResults=100)
    response = request.execute()

    compromised_count = 0
    for activity in response.get("items", []):
        actor_email = activity.get("actor", {}).get("email")
        for event in activity.get("events", []):
            event_name = event.get("name")
            params = {p["name"]: p.get("value") for p in event.get("parameters", [])}
            client_id = params.get("client_id")
            app_name = params.get("app_name", "Unknown App")
            scopes = params.get("scope_data", "")

            if client_id in KNOWN_SUSPICIOUS_CLIENT_IDS:
                compromised_count += 1
                threat_note = KNOWN_SUSPICIOUS_CLIENT_IDS[client_id]
                logging.error(f"CRITICAL FINDING: User {actor_email} authorized malicious application!")
                logging.error(f"  App Name: {app_name} | Client ID: {client_id}")
                logging.error(f"  Threat Details: {threat_note}")
                logging.error(f"  Action Required: Immediately revoke token for {actor_email}")

    logging.info(f"Audit completed. Found {compromised_count} high-severity findings.")

if __name__ == "__main__":
    if len(sys.argv) < 3:
        print(f"Usage: {sys.argv[0]} <service_account.json> <admin@company.com>")
        sys.exit(1)
    audit_domain_tokens(sys.argv[1], sys.argv[2])
```

14.2 Bash Emergency Token Revocation & Vercel API Secret Audit Tool

This automated bash playbook revokes compromised OAuth tokens at Google's endpoint and audits a Vercel organization for unencrypted ("Standard") environment variables:

BASH

```bash
#!/usr/bin/env bash
# emergency_token_revocation_and_vercel_audit.sh
# Automates token revocation and sweeps Vercel projects for unencrypted environment variables.
set -euo pipefail

OAUTH_TOKEN="${1:-}"
VERCEL_TOKEN="${2:-}"

if [[ -z "$OAUTH_TOKEN" || -z "$VERCEL_TOKEN" ]]; then
    echo "Usage: $0 <GOOGLE_OAUTH_TOKEN> <VERCEL_API_BEARER_TOKEN>"
    exit 1
fi

echo "[+] Step 1: Initiating immediate token revocation against Google Authorization Server..."
HTTP_STATUS=$(curl -s -o /dev/null -w "%{http_code}" -X POST     -d "token=${OAUTH_TOKEN}"     "https://oauth2.googl

if [[ "$HTTP_STATUS" == "200" ]]; then
    echo "    [SUCCESS] Token successfully revoked at Google Authorization Server."
else
    echo "    [WARNING] Revocation endpoint returned HTTP ${HTTP_STATUS}. Verify token status manually."
fi

echo "[+] Step 2: Auditing Vercel projects for unmasked environment variables..."
PROJECTS_PAYLOAD=$(curl -s -H "Authorization: Bearer ${VERCEL_TOKEN}"     "https://api.vercel.com/v9/projects?limit=

PROJECT_IDS=$(echo "$PROJECTS_PAYLOAD" | grep -o '"id":"prj_[^"]*"' | cut -d'"' -f4 || true)

if [[ -z "$PROJECT_IDS" ]]; then
    echo "    [-] No projects found or invalid Vercel API token."
    exit 0
fi

TOTAL_EXPOSED=0
for PID in $PROJECT_IDS; do
    ENV_DATA=$(curl -s -H "Authorization: Bearer ${VERCEL_TOKEN}"         "https://api.vercel.com/v9/projects/${PID}

    PLAIN_COUNT=$(echo "$ENV_DATA" | grep -o '"type":"plain"' | wc -l || true)
    SENSITIVE_COUNT=$(echo "$ENV_DATA" | grep -o '"type":"sensitive"' | wc -l || true)

    if [[ "$PLAIN_COUNT" -gt 0 ]]; then
        echo "    [ALERT] Project ${PID}: ${PLAIN_COUNT} plaintext secrets detected! (${SENSITIVE_COUNT} encrypted)"
        TOTAL_EXPOSED=$((TOTAL_EXPOSED + PLAIN_COUNT))
    else
        echo "    [OK] Project ${PID}: All ${SENSITIVE_COUNT} secrets properly encrypted."
    fi
done

echo "[+] Audit Complete. Total unmasked production secrets identified: ${TOTAL_EXPOSED}"
```

Chapter 15: Regulatory Compliance, Legal Implications & References

15.1 Legal & Regulatory Implications

The exposure of customer environment variables carries extensive compliance and liability ramifications across multiple international regulatory frameworks:

- Under GDPR, organizations processing EU citizen data must report personal data breaches to supervisory authorities within of becoming aware of the incident. Because customer database connection strings and Supabase service-role keys allowed direct exfiltration of production databases containing Personally Identifiable Information (PII), affected enterprise customers faced mandatory disclosure obligations and potential administrative fines of up to 4% of annual global turnover under Article 83.
- For publicly traded companies whose production credentials were compromised, the SEC's rules mandate public disclosure within of determining that an incident is material. Evaluating materiality required organizations to quantify potential financial harm resulting from compromised payment gateways, intellectual property theft, and infrastructure tampering.
- The incident directly violated core SOC 2 criteria, specifically (access controls prohibiting unauthorized logical access) and (logical boundary segregation and encryption of sensitive customer data at rest). Vercel's subsequent architectural remediation directly addressed these control gaps by mandating envelope encryption and eliminating plaintext secret serialization.

15.2 Primary Standards & Academic Citations

1. *The OAuth 2.0 Authorization Framework*. RFC 6749, Internet Engineering Task Force (IETF).
2. *The OAuth 2.0 Authorization Framework: Bearer Token Usage*. RFC 6750, Internet Engineering Task Force (IETF).
3. *OAuth 2.0 Demonstrating Proof-of-Possession at the Application Layer (DPoP)*. RFC 9449, Internet Engineering Task Force (IETF).
4. *Zero Trust Architecture*. NIST Special Publication 800-207, National Institute of Standards and Technology.
5. *Recommendation for Key Management: Part 1 – General*. NIST Special Publication 800-57 Part 1 Rev. 5, National Institute of Standards and Technology.
6. *MITRE ATT&CK Enterprise Matrix (Version 14)*. Bedford, MA: The MITRE Corporation.
7. *Continuous Access Evaluation Profile 1.0 (CAEP)*. OpenID Shared Signals Working Group.
8. *Regulation (EU) 2016/679 (General Data Protection Regulation)*. Official Journal of the European Union.

Authored by Cloud Infrastructure Security & Threat Research Working Group. Document Reference CS-2026-VCL-01.

Published for engineering post-mortem review, incident response training, and cloud security curriculum.